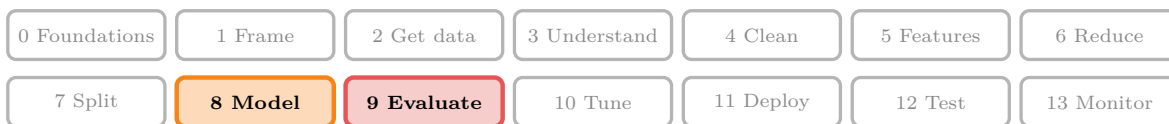


Polynomial Regression

Contents

1. Overview	2
2. A curved pattern	3
3. Adding powers as new features	4
4. Choosing the degree	6
5. More than one feature	7
6. Summary	9
7. Sources	9
8. Key terms	9

Where this fits: Pipeline map steps 8 (Model), 9 (Evaluate). See the [Course map](#).



- **Builds on:** Train-test split ([Note ML-012](#)); Multiple linear regression ([Note ML-052](#)).
- **Leads to:** Bias-variance trade-off ([Note ML-061](#)); Regularisation ([Note ML-062](#)); Logistic regression ([Note ML-069](#)); Hyperparameter tuning ([Note ML-080](#)); Boosting ([Note ML-095](#)); Bagging ([Note ML-099](#)).
- **Compare with:** Decision surface and boundary ([Note ML-085](#)); Kernel trick ([Note ML-089](#)).

1. Overview

Key point: Polynomial regression fits curves by adding powers of the feature (x^2 , x^3 , ...) as new features and then running ordinary linear regression on them.

Linear regression fits a straight line (or flat plane). Real relationships are often curved: the output may first fall and then rise, or grow faster and faster. A straight line misses such patterns.

A **feature** (G-772) is an input variable (one column of the data table), an **observation** (G-1374) is one record (one row), and the **target** (G-1949) is the output we predict.

Polynomial regression (G-1515) handles curves without a new algorithm. Polynomial regression creates new features from powers of the existing ones and gives them to plain **linear regression** (G-1094), the way a straight ruler can trace a curve if we feed it bent graph paper. The **degree** (G-577) is the highest power used.

Figure 1 shows the trick on this Note's data. Each point is drawn against x and against a new feature, x^2 . In that space the points lie close to a flat plane, and fitting a flat plane is exactly what linear regression does. The black curve is where the data can sit on the plane: seen from the side, it is the U-shaped fit.

with the new feature x^2 , the curve lies on a flat plane:

$$\hat{y} = 1.92 + 1.04 x + 0.82 x^2$$

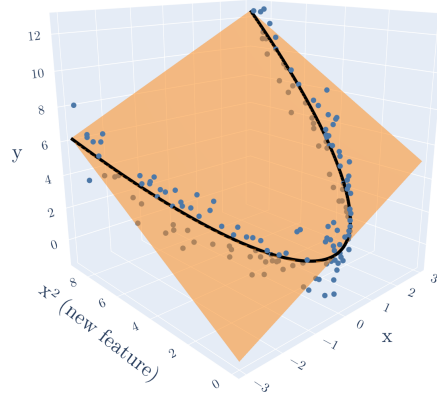


Figure 1: The 160 training points of Section 2 plotted against x and the new feature x^2 , with the target upwards. They lie close to the flat plane $\hat{y} = 1.92 + 1.04x + 0.82x^2$, which linear regression fits; the black curve along the plane is the fitted U-shape.

2. A curved pattern

Key point: On data that follows a parabola, a straight line reaches only R^2 0.38.

The example data has 200 points that follow

$$y = 0.8x^2 + 0.9x + 2 + \text{noise}$$

for x between -3 and 3 : a U-shaped curve with some random scatter. Fitted with ordinary linear regression, the best straight line scores only $R^2 = 0.38$ on the test points. The line cannot bend, so it misses the low middle and both high ends (Figure 2). The misses form a pattern: the points sit above the line at both ends and below it in the middle. A residual pattern like this is the sign of a curved relationship (assumption 1 of the [linear regression assumptions](#) Note).

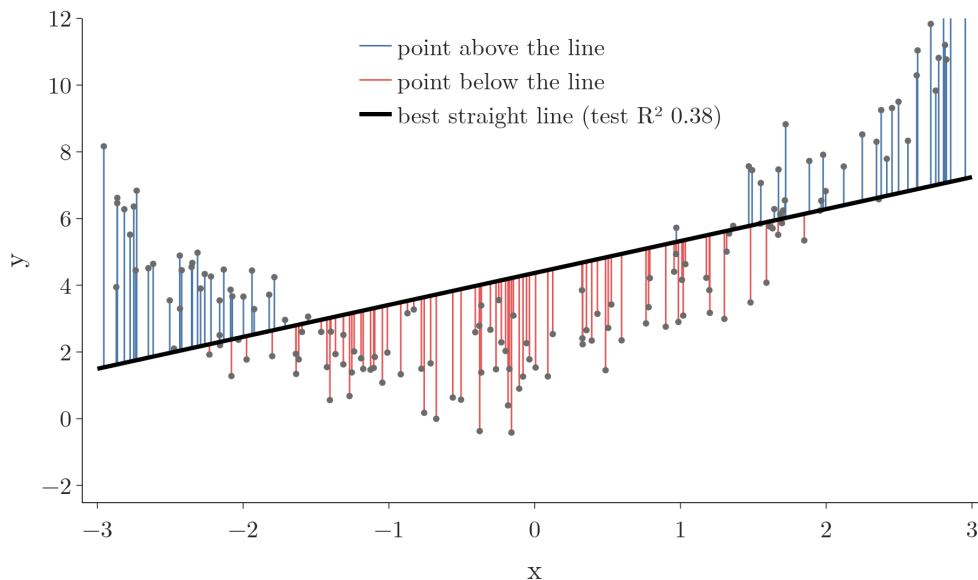


Figure 2: The best straight line on the 160 training points (test R^2 0.38). Blue sticks: points above the line, mostly at both ends. Red sticks: points below it, mostly in the middle.

3. Adding powers as new features

Key point: For degree 2, each observation gets a new feature holding x^2 . Linear regression then finds one coefficient per feature.

The trick: for each observation, compute x^2 and add it as a new column of the table (Figure 3).

x			
−0.89	PolynomialFeatures →		
1.50			
2.10			
	1	x	x^2
−0.89	1	−0.89	0.79
1.50	1	1.50	2.25
2.10	1	2.10	4.41

a new column x^2 (and a column of 1s); the model stays linear in its coefficients: $\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$

Figure 3: PolynomialFeatures (G-1516) turns one column into 1, x and x^2

With the features x and x^2 , linear regression fits

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$$

The fitted model is a curve in x , but it is still a straight-line combination of its coefficients: each coefficient just multiplies a feature. This is what “**linear** regression” means here: linear in the coefficients, not in x . With made-up numbers $\beta_0 = 1$, $\beta_1 = 2$, $\beta_2 = 3$ the prediction at $x = 2$ is

$$1 + 2 \times 2 + 3 \times 2^2 = 1 + 4 + 12 = 17$$

and the curve through $x = 0, 1, 2$ gives 1, 6, 17: bending upwards, though every coefficient only multiplies a feature once. So the usual linear regression machinery (OLS or gradient descent) finds β_0 , β_1 and β_2 unchanged.

Linearity in the coefficients is why polynomial regression is called “linear”: **linear** refers to the coefficients, not to the shape of the curve in x .

Python: Degree-2 polynomial regression.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
```

```
poly = PolynomialFeatures(degree=2, include_bias=True)
X_train_p = poly.fit_transform(X_train)    # [x] -> [1, x, x^2]
X_test_p = poly.transform(X_test)
```

```
lr = LinearRegression().fit(X_train_p, y_train)
lr.score(X_test_p, y_test)    # R^2 0.83
lr.coef_, lr.intercept_      # [0, 1.04, 0.82], 1.92
```

`include_bias=True` (G-929) adds the column of 1s; `LinearRegression` fits its own intercept anyway, so that column gets coefficient 0 and either setting works.

Two things to get right: the powers are made from the features only, never from the target; and `fit_transform` goes on the training features while the test features get the same `transform`, as with any preprocessing step.

With the x^2 feature, the test R^2 jumps from 0.38 to 0.83. The learned equation, $\hat{y} = 1.92 + 1.04x + 0.82x^2$, is close to the true $2 + 0.9x + 0.8x^2$. The remaining difference comes from the noise: refitting on the same training observations with the noise removed gives exactly $2 + 0.9x + 0.8x^2$ (notebook).

4. Choosing the degree

Key point: Too low a degree underfits; too high a degree bends to fit the noise and overfits. Training R^2 keeps rising with the degree, but test R^2 falls.

Higher degrees add more features, x^3 , x^4 and so on, and the curve can bend more. Figure 4 fits three degrees to only 25 training points, then scores them on 200 new test points.

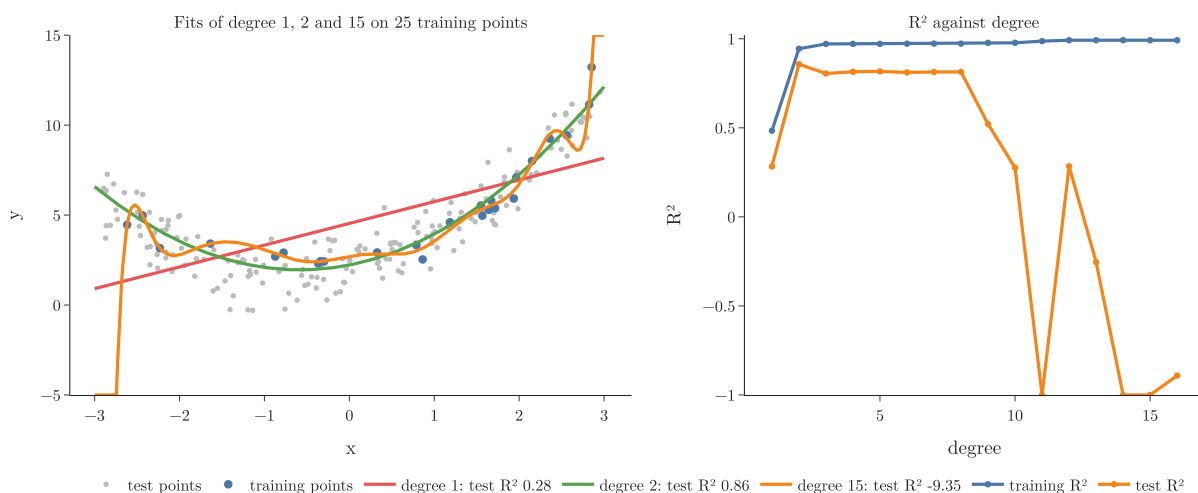


Figure 4: Polynomial fits of degree 1, 2 and 15, and R^2 against the degree

Degree	Training R^2	Test R^2	
1	0.48	0.28	underfits (G-2035): too simple to follow the curve
2	0.94	0.86	right shape: matches how the data was made
6	0.97	0.81	starting to fit noise
10	0.98	0.28	overfits
15	0.99	-9.35	overfits badly: wild swings between and beyond the points

The right panel shows the pattern. **Training R^2** rises with every extra degree. The maths guarantees it: a degree $(d + 1)$ model contains every degree d curve (set the new coefficient to 0), so its best training error can only be equal or smaller. **Test R^2** peaks at degree 2 and then collapses: the high-degree curve has learned the noise of these 25 points, not the pattern (the challenges and fitting Notes called this **overfitting**, G-1429).

Figure 5 runs through all fifteen degrees one at a time. Watch the curve on the left: up to degree 8 it stays close to the U-shape, then it starts to wiggle between the training points and shoot off at the edges, and the orange test R^2 on the right falls away while the blue training R^2 stays near the top.

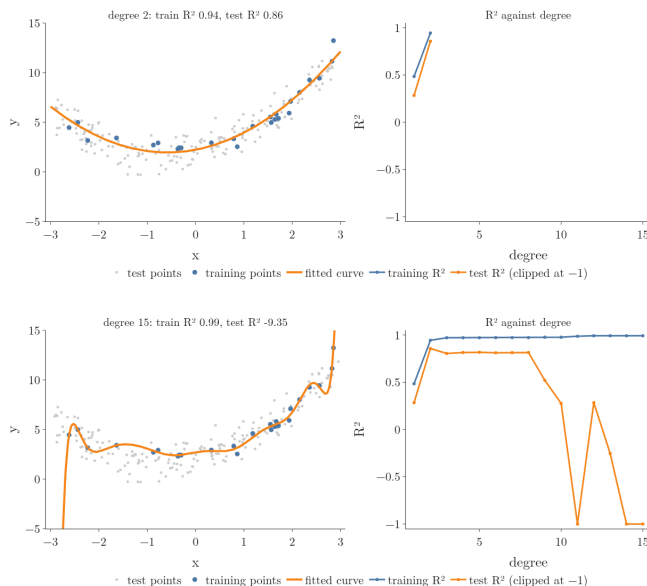


Figure 5: Polynomial fits of degree 1 to 15 on the 25 training points, one degree per frame. Left: the fitted curve against the training points (blue) and 200 test points (grey). Right: training and test R^2 so far; test R^2 is clipped at -1 .

The degree is a **hyperparameter** (G-910), chosen by comparing scores on data not used for training, as here, or with cross-validation (a later Note).

Extra: A degree-15 polynomial has terms up to x^{15} ; with $x = 3$ that is about 14 million. Columns on such different scales make the fit sensitive to rounding errors. The **condition number** (G-441) measures that sensitivity: the larger it is, the more small rounding errors can change the answer (Goodfellow §4.2). For the 25 training points it is 4.5×10^8 for the raw columns and 1.1×10^6 after **StandardScaler**, so Figure 4 uses a pipeline of **PolynomialFeatures**, **StandardScaler** and **LinearRegression**.

5. More than one feature

Key point: With several features, **PolynomialFeatures** also adds products of features, such as xy . The number of features grows very fast with the degree.

With two features x and y , degree 2 creates all terms of total power up to 2:

$$1, x, y, x^2, xy, y^2$$

The product xy is an **interaction term** (G-959): it lets the effect of x depend on the value of y . In maths: if $\hat{z} = \dots + \beta_x x + \beta_{xy} xy$, a rise of 1 in x changes $\hat{z}$ by $\beta_x + \beta_{xy} y$, which depends on y . With made-up numbers $\beta_x = 1$ and $\beta_{xy} = 0.5$: at $y = 0$ a rise of 1 in x changes $\hat{z}$ by $1 + 0.5 \times 0 = 1$, and at $y = 4$ by $1 + 0.5 \times 4 = 3$. The fitted model is a curved surface instead of a flat plane.

Figure 6 shows data made from $z = x^2 + y^2 + 0.2x + 0.2y + 0.1xy + 2$ plus noise. A plane reaches $R^2 = 0.61$; a degree-2 surface reaches 0.98.

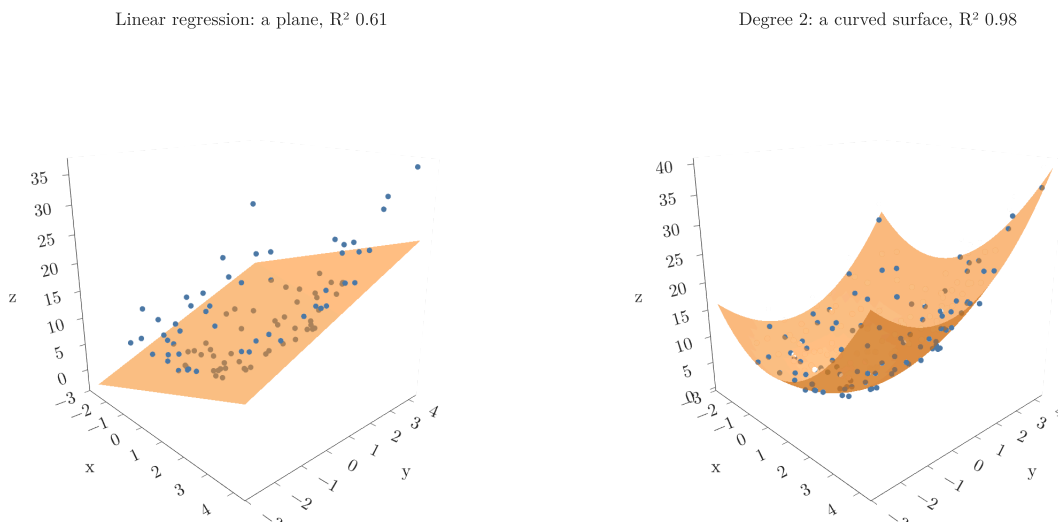


Figure 6: A plane vs a degree-2 surface on curved 3D data

The number of new features grows quickly. With 2 features, degree 2 gives 6; degree 30 gives 496. With many features and a high degree, the new features quickly outnumber the observations. Once there are as many features as observations, least squares passes exactly through every training point, whatever the true relationship, and such a model does very poorly on new data (ISL §6.4.2).

Extra: To put two 1-D arrays side by side as columns, use `np.c_[x, y]` or `np.column_stack([x, y])`. A shortcut sometimes seen, `np.array([x, y]).reshape(100, 2)`, does not do this: it fills the rows with 100 values of `x` first and then of `y`, so most rows pair unrelated numbers. The model still runs, but on scrambled data. Checking `X[:3]` against `x[:3]` and `y[:3]` catches it.

6. Summary

Degree	Features from one original feature	Behaviour on the example
1	x	straight line, underfits (test R^2 0.28)
2	x, x^2	follows the curve (test R^2 0.86)
15	x to x^{15}	overfits (test R^2 -9.35)

- Polynomial regression = new power features + ordinary linear regression.
- Polynomial regression is “linear” in the coefficients, so OLS and gradient descent work unchanged.
- `PolynomialFeatures(degree=d)` creates the new features; with several features it adds interaction terms too.
- The degree controls flexibility: too low underfits, too high overfits. Choose it on held-out data.

7. Sources

Built from

- CampusX, “Polynomial Regression | Machine Learning”, YouTube, <https://www.youtube.com/watch?v>

Other references

- **Goodfellow**: I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016 (deeplearningbook.org). §4.2 (poor conditioning, condition number).
- **ISL**: James, Witten, Hastie, Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021. §6.4.2 (what goes wrong in high dimensions).

8. Key terms

Term	Meaning
Feature	An input variable: one column of the data table
Observation	One record: one row of the data table
Target	The output we predict
Polynomial regression	Linear regression on powers (and products) of the features, to fit curves
Degree	The highest power used in the polynomial
<code>PolynomialFeatures</code>	scikit-learn transformer that creates the power and product columns
<code>include_bias</code>	<code>PolynomialFeatures</code> setting that adds a column of 1s

Term	Meaning
Interaction term	A product of two features, such as xy , that lets one feature's effect depend on another
Underfitting	A model too simple to capture the pattern; poor on training and test data
Overfitting	A model so flexible that it learns the noise; good on training data, poor on test data